



Tema 1:
Análisis de Algoritmos

Estructura de Datos (2do. año)
Carrera Analista de Sistemas/Licenciatura en Sistemas

Universidad Nacional de la Patagonia Austral
Unidad Académica Río Gallegos

¿Qué es un buen algoritmo?

Dados dos algoritmos diferentes que resuelven el mismo problema:

¿cuál elegimos? ¿Cómo los comparamos?

Respuesta: El que consume menos tiempo es el mejor.

INCORRECTO.

Se debe hacer un análisis de algoritmos (AA) y en base a estos elegiremos el mejor, el mas eficiente.

Dados dos algoritmos, el más eficiente es el que consume o requiere menos **trabajo-recursos**.

Aunque el tiempo es una consideración importante.

¿Cómo se mide el trabajo que realiza un algoritmo?

1. Tomar el tiempo de ejecución
2. Contar la cantidad de líneas del algoritmo

Ambos criterios son INCORRECTOS.

El tiempo depende de la computadora, S.O., las líneas del lenguaje y estilo de programación.

- Es necesaria una medida más general, que determine la eficiencia de un algoritmo más allá de la computadora donde se ejecuta y del lenguaje de programación.
- El objetivo del AA es determinar T que es una función del trabajo que hace el algoritmo y de allí determinar O (orden de magnitud).

$$\begin{aligned} T &= f(N) \\ f(N) &= 4N^2 + 5N + 3 \\ f(N) &= O(N^2) \end{aligned}$$

Donde n es el tamaño del problema (por lo general entrada) y $O(n^2)$ es la medida que se usa para comparar algoritmos.

Si el algoritmo X tiene $O(n^2)$ y el algoritmo Y tiene $O(n^3)$, X es más eficiente que Y , puesto que $O(n^2) < O(n^3)$, es decir X requiere menos trabajo y recursos que Y .

Para hallar O , dada una f , se toma el término de “mayor orden” (operación fundamental) y se desprecian las constantes y términos de menor orden de la expresión.

$$f(n) = 4n^2 + 5n + 3$$

se desprecia la constante 4 del primer término, y el segundo y tercer término por ser de menor orden. El término de mayor orden en $f(n)$ es el que crece más rápido, en este caso $4n^2$. $4n^2$ siempre va a ser mayor que $5n$, para cualquier valor de n excepto 1 y 2.

n	$4n^2$	$5n$
0	0	0
1	4	5
2	16	20
3	36	15

La mayoría de las veces, una operación dominará el algoritmo de tal forma que las otras operaciones carecen de importancia.

MODELO

Para AA se define un modelo. Nuestro modelo radica en una computadora normal en la cual se ejecutan instrucciones en modo secuencial.

El modelo tiene un repertorio de instrucciones sencillas (+,-,*,-,mod, comparaciones, asignación, lectura y escritura) y tarda exactamente una unidad de tiempo en hacer cualquiera de ellas.

También suponemos memoria infinita.

Es obvio que esto no es tan así, en la realidad no todas las operaciones tardan lo mismo.

En particular una operación de lectura o escritura a disco, demora más que una adición.

También al suponer memoria infinita nunca hemos de preocuparnos por falta de paginas, lo cual suele ser un problema real.

¿Que analizar?

El recurso más importante a analizar es el tiempo vs. factores que afectan al tiempo de ejecución:

- Compiladores
- La computadora
- El algoritmo
- La entrada

Ejemplo

Factorial(n): entero

1. $f \leftarrow 1$
2. mientras $n \geq 1$ hacer
3. $f \leftarrow f * n$
4. $n \leftarrow n - 1$
5. factorial $\leftarrow f$

Si $n = 1000$ tardará más que si $n = 10$. Es decir el tamaño del problema afecta al tiempo de ejecución. Por ello T es una $f(n)$ (función del tamaño)

T para factorial

Línea 1: requiere una unidad de tiempo (una asignación) = 1

Líneas 3 y 4: requiere dos unidades de tiempo cada una (una asignación y un calculo aritmético) = 4

Línea 5: requiere una unidad de tiempo (una asignación) = 1

Las líneas 3 y 4 están dentro de un bucle que se ejecuta n veces = 4n

Línea 2: requiere una unidad de tiempo (una comparación) que se ejecuta n+1 veces = n+1

$$f(n) = 1 + n+1 + 4n + 1$$

$$f(n) = 3 + 5n$$

$$f(n) = O(n)$$

la operación mas fuerte (requiere más trabajo) es el bucle.

Tiempo de ejecución T(n) de un algoritmo

- Para medir un T(n) se realiza a través del número de operaciones elementales.
- Una operación elemental puede ser:
 - Operación aritmética (+, -, /, *, % (MOD)).
 - Asignación a una variable {=}
 - Operaciones aritméticas con asignación {+=, -=, *=, /=, ++, --}
 - Cuentan como 2 OE ya que 1 es por la operación y otra por la asignación.
 - Llamada/retorno a una función.
 - Igualdades y Desigualdades {=, !=}.
 - Comparaciones lógicas {<, >, <=, >=, ||, &&}.
 - Acceso a una estructura de datos (arreglo, matriz, etc.).
 - Tiempo de ejecución
 - No es tiempo físico
 - Es el número de operaciones elementales que se llevan a cabo en el algoritmo.

Cálculo de $T(n)$

- Es importante tener en cuenta lo siguiente:
 - El tiempo de una OE es, por definición, de orden $O(1)$.
 - El tiempo de ejecución de una secuencia de instrucciones
 - se calcula sumando los tiempos de ejecución de cada una de las instrucciones.
 - Las declaraciones de variables, funciones, constantes, entre otras, no se considerarán al momento del cálculo.
 - Excepto aquellas declaraciones de variables que incluyan la inicialización.
 - El uso de funciones de E/S, tales como `printf()` (en C), `System.out...` (java), implicaran el costo de 1 OE.
 - Si alguna de ellas implica el uso de variables, se sumará 1 OE por el uso de cada una de ellas.

NOTACIÓN ASINTOTICA (O-grande)

El interés es hacer AA en problemas de gran tamaño. Ej. si siempre se va a trabajar con un array de 5 elementos (pequeño) no es necesario hacer AA, pero si el array ha de tener miles de elementos entonces hacemos AA.

Se habla del trabajo (tiempo) como una función del tamaño del problema (datos de entrada)

Se puede expresar una aproximación de esta función usando una notación matemática llamada “Orden de Magnitud” o “notación O grande”.

El orden de magnitud de una función es la misma que el orden (exponente) del término de la función que crece más rápido respecto del tamaño del problema.

$$f(n) = n^4 + 100n^2 + 10n + 50$$

$$O(n^4)$$

Despreciamos constantes y términos de menor orden.

El orden de magnitud no dice cuantos nanosegundos tardará la solución en ejecutarse sobre la computadora. La información O-grande nos permite comparar algoritmos.

Ordenes de magnitud comunes

función	nombre
C	Constante
N	Lineal
N ²	Cuadrática
N ³	Cúbica
Log n	Logarítmica
Log ² n	Logarítmica cuadrática
N log n	
2 ⁿ	exponencial

Función	Ejemplo
Un tiempo constante es referido como $O(1)$, significa que el tiempo requerido por la operación no cambia.	Operaciones simples (asignación, comparación, etc.)
Un algoritmo $O(n)$ se conoce como lineal	Examinar los n elementos del array
Un algoritmo $O(\log n)$ trabaja mas que $O(1)$ pero menos que $O(n)$,	Acceso a las hojas de un árbol binario.
Un algoritmo $O(n^2)$ cuadrática trabaja mas que $O(n)$	Recorrido de matrices
Un algoritmo $O(n^3)$ cúbico trabaja mas que $O(n^2)$	Algunos algoritmos de grafos
Los algoritmos de tiempo exponencial $O(n^2)$ son realmente costosos, son los peores.	

Comparación de razones de crecimiento

N	Log N	N log N	N^2	N^3	2^N
1	0	1	1	1	2
2	1	2	4	8	4
4	2	8	16	64	16
8	3	24	64	512	256
16	4	64	256	4096	65536
32	5	160	1024	32768	2147483648
64	6	384	4096	262144	-
128	7	896	16384	2097152	-
256	8	2048	65536	16777216	-

Reglas generales

Regla 1: Sentencias simples.

- El $T(n)$ de ejecución de una sentencia, es igual a la suma de las OE que posee dicha sentencia:

1) `int numero, a = 5, b = 7, c = 3, d = 2;` ← 4 OE

2) `numero = a * c + b - d + 4;` ← 5 OE

Entonces, realizar la suma de las operaciones elementales:

$$4 \text{ OE (Línea 1)} + 5 \text{ OE (Línea 2)} = 9 \text{ OE}$$

$$T(n) = 9 \text{ OE}$$

$$T(n) = 9 O(1)$$

Complejidad constante

Regla 2: condicional "if"

- El $T(n)$ de ejecución "if", es el costo de las instrucciones que se ejecutan condicionalmente, más el tiempo para evaluar la condición.
- El $T(n)$ sería: $T(\text{if}) = T(\text{condición}) + T(\text{cuerpo})$
- Ejemplo:

```

1) Scanner scanner = new Scanner(System.in); ← 2 OE
2) int a, b, c, d;
3) System.out.println("Ingrese el valor de a, b y c: "); ← 1 OE
4) a = scanner.nextInt();
5) b = scanner.nextInt(); } 6 OE
6) c = scanner.nextInt();
7) if(a > b) { ← 1 OE
8)     a = 2 * b + c;
9)     c = a / 2; } 7 OE
10)    b = a + c;
11) }
12) d = a + b = c; ← 3 OE
13) System.out.println("El valor de d es: ", d); ← 1 OE
  
```

Análisis de Algoritmos

Estructura de Datos

23

Regla 2: condicional "if"

Entonces, realizar la suma de las operaciones elementales:

$$T(n) = 2 \text{ OE (L1)} + 1 \text{ OE (L3)} + 6 \text{ OE (L4-5-6)} + 1 \text{ OE (L7)} + 7 \text{ OE (L8-9-10)} + 3 \text{ OE (L12)} + 1 \text{ OE (L13)} = 21 \text{ OE}$$

$$T(n) = 21 \text{ OE}$$

$$T(n) = O(1)$$

Complejidad constante

Análisis de Algoritmos

Estructura de Datos

24

Regla 3: condicional "if - else"

- El $T(n)$ de ejecución "if - else" es el costo para evaluar la condición más el mayor entre los tiempos necesarios para ejecutar las proposiciones cuando la condición es verdadera y el tiempo de ejecución de las proposiciones cuando la condición es falsa.
- $T(\text{if - else}) = T(\text{condición}) + \text{MAX}(T(\text{cuerpo}_{\text{si}}), T(\text{cuerpo}_{\text{no}}))$
- Ejemplo:

```

1) Scanner scanner = new Scanner(System.in);
2) int a, b, c, d;
3) System.out.println("Ingrese el valor de a, b y c: ");
4) a = scanner.nextInt();
5) b = scanner.nextInt();
6) c = scanner.nextInt();
7) If (a > b) { ← 1 OE
8)     a = 2 * b + c;
9)     c = a / 2;
10)    b = a + c;
11) } else {
12)     a = 2 * b + c * a;
13)     c = a / 2;
14) }
15) d = a + b - c;
16) System.out.println("El valor de d es: ", d);

```

Diagrama de anotaciones de operaciones elementales (OE):

- Una flecha roja apunta a la línea 7 con la etiqueta "1 OE".
- Un corchete rojo agrupa las líneas 8, 9 y 10 con la etiqueta "7 OE".
- Un corchete rojo agrupa las líneas 12 y 13 con la etiqueta "6 OE".

Entonces, realizar la suma de las operaciones elementales:

$$T(n) = 2 \text{ OE (L1)} + 1 \text{ OE (L3)} + 6 \text{ OE (L4-5-6)} + 1 \text{ OE (L7)} + 7 \text{ OE (L8-9-10)} + 3 \text{ OE (L15)} + 3 \text{ OE (L15)} + 1 \text{ OE (L16)} = 24 \text{ OE}$$

$$T(n) = 24 \text{ OE}$$

$$T(n) = O(1)$$

Complejidad constante

Regla 4: Llamados a funciones.

- El $T(n)$ de ejecución de una función $F(P_1, P_2, \dots, P_n)$ es igual a:
 - 1 OE (por la llamada), más el tiempo de evaluación de los parámetros $P_1, P_2, \dots, P_n$, es decir, el tiempo que se emplea en obtener el valor de cada parámetro, más el tiempo que tarde en ejecutarse el cuerpo de la función F , esto es:
- $T(\text{llamada a la función}) = 1 \text{ OE} + T(P_1) + T(P_2) + \dots + T(P_n) + T(\text{cuerpo}_{\text{función}})$.
- Ejemplo (lenguaje C):

```

1) int suma (int num1, int num2){
2)     int num3;
3)     num3 = num1 + num2;
4)     return num3;
5) }
6) int main(){
7)     int numero1 = 4, numero2 = 5, resultado;
8)     resultado = suma (numero1, numero2 * 3);
9)     printf("El resultado es: %d", resultado);
10) return 0;
11) }
```

Diagrama de anotaciones de tiempo de ejecución (OE):

- En la función `suma`:
 - Las líneas 2 y 3 están agrupadas por una llave roja y etiquetadas como **2 OE**.
 - La línea 4 está marcada con una flecha roja y etiquetada como **1 OE**.
- En la función `main`:
 - Las líneas 7, 8 y 9 están agrupadas por una llave roja y etiquetadas como **7 OE**.
 - La línea 10 está marcada con una flecha roja y etiquetada como **1 OE**.

Análisis de Algoritmos

Estructura de Datos

27

Regla 4: Llamados a funciones.

$$T(n) = 2 \text{ OE } (L_7) + \left(\begin{array}{l} 1 \text{ OE}(\text{llamada}) + \\ 1 \text{ OE}(\text{operación}_{\text{producto}}) + \\ 3 \text{ OE}(\text{cuerpo}_{\text{función}}) + \\ 1 \text{ OE}(\text{asignación}) \end{array} \right) (\text{Todas en } L_8) + 2 \text{ OE } (L_9) + 1 \text{ OE } (L_{10})$$

$$T(n) = 11 \text{ OE}$$

$$T(n) = 2 \text{ OE } (L_7) + \left(\begin{array}{l} 1 \text{ OE}(\text{llamada}) + \\ 1 \text{ OE}(\text{operación}_{\text{producto}}) + \\ 3 \text{ OE}(\text{cuerpo}_{\text{función}}) + \\ 1 \text{ OE}(\text{asignación}) \end{array} \right) (\text{Todas en } L_8) + 2 \text{ OE } (L_9) + 1 \text{ OE } (L_{10})$$

$$T(n) = 11 \text{ OE}$$

Análisis de Algoritmos

Estructura de Datos

28

Regla 5: Ciclos o repeticiones. (1)

- El $T(n)$ de ejecución de un ciclo es
 - el tiempo de ejecución de las instrucciones que están en el interior del ciclo por el número de iteraciones,
 - y también se tienen en cuenta las operaciones que realizan modificaciones en alguna variable para que la condición del bucle finalice, las evaluaciones que se realizan sobre la condición para saber si finaliza o no.
- A continuación tenemos la estructura básica de un "for" y un "while":
 - 1) for(inicialización; comparación; incremento o decremento){
 - 2) instrucciones;
 - 3) }
- Análogamente también podría ser:
 - 1) inicialización;
 - 2) while (comparación){
 - 3) instrucciones;
 - 4) incremento o decremento;
 - 5) }

Regla 5: Ciclos o repeticiones. (2)

- De esta forma se tiene:
 - La inicialización se realiza una única vez.
 - La comparación se realiza N veces
 - siendo N la cantidad de comparaciones hasta que la condición es falsa
 - la comparación realmente se realiza $(N + 1)$ veces, el $+1$ es la última comparación dónde es falsa y no realiza el bucle.
 - Las instrucciones se realizan N veces.
 - El incremento o decremento se realiza N veces.
 - Entonces se puede establecer que $T(n)$ es:

$$T(n) = T(\text{inicialización}) + T(\text{comparación}) + N * \left(\begin{array}{l} T(\text{instrucciones}) + \\ T(\text{comparación}) + \\ T(\text{incremento o decremento}) \end{array} \right)$$

Regla 5: Ciclos o repeticiones. (3)

- Ejemplo:

- 1) Scanner scanner = new Scanner(System.in); ← 2 OE
- 2) int i, N, suma = 0; ← 1 OE
- 3) System.out.println("Ingrese el valor de N: "); ← 1 OE
- 4) N= scanner.nextInt(); ← 2 OE
- 5) for(^{1 OE}i = 0; ^{1 OE}i < N; ^{2 OE}i++)
- 6) suma += i * 2; ← 3 OE
- 7) System.out.println("El resultado es: ", suma); ← 1 OE

Regla 5: Ciclos o repeticiones. (4)

- Realizando la suma de todas las operaciones elementales tenemos:

$$T(n) = 2 \text{ OE (L1)} + 1 \text{ OE (L2)} + 1 \text{ OE (L3)} + 2 \text{ OE (L4)} + \left(N * \begin{pmatrix} 1 \text{ OE} + \\ 1 \text{ OE} + \\ 3 \text{ OE (L6)} + \\ 1 \text{ OE} + \\ 2 \text{ OE} \end{pmatrix} \right) + 1 \text{ OE (L7)}$$

(L5for)

$$T(n) = 7 \text{ OE} + 2 \text{ OE} + N * 6 \text{ OE}$$

$$T(n) = 6 \text{ OE} * N + 9 \text{ OE}$$

$$T(n) = 6N + 9 \rightarrow O(n)$$

Complejidad lineal.

Regla 5: Ciclos o repeticiones. (5)

- La estructura repetitiva "for" realiza:

- Suma de las operaciones elementales internas como la comparación, el incremento y las instrucciones, la cantidad de veces como la comparación sea verdadera
- Es decir:

$$T(n) = T(\text{inicialización}) + T(\text{comparación}) + \sum_{\text{valor inicial}}^{\text{valor final}} \left(\begin{array}{l} T(\text{instrucciones}) + \\ T(\text{comparación}) + \\ T(\text{incremento o decremento}) \end{array} \right)$$

- Para calcular el valor de sumatoria, se aplica la siguiente formula:

- valorFinal - valorInicial + 1**
- valorFinal: el ultimo valor valido de N y que ingresa al bucle

- Por lo tanto, el cálculo de T(n) del "for" es:

$$T(n) = T(\text{inicialización}) + T(\text{comparación}) + (\text{Valor}_{\text{final}} - \text{Valor}_{\text{inicial}} + 1) * (T(\text{instrucciones}) + T(\text{comparación}) + T(\text{incremento o decremento}))$$

Regla 6: Ciclos Anidados.

- El T(n) de ejecución de dos o más ciclos anidados sigue la misma regla de un único ciclo, siendo que el ciclo interno formará parte de las instrucciones del ciclo externo, y así sucesivamente:

$$T(n) = T(\text{inicialización}) + T(\text{comparación}) + \sum_{\text{valor inicial}}^{\text{valor final}} \left(\begin{array}{l} T(\text{instrucciones o } \text{for}_{\text{interno}}) + \\ T(\text{comparación}) + \\ T(\text{incremento o decremento}) \end{array} \right)$$

Regla 6: Ciclos Anidados.

- 1) Scanner scanner = new Scanner(System.in); ← 2 OE
- 2) int i, j, N, suma = 0; ← 1 OE
- 3) System.out.println("Ingrese el valor de N: "); ← 1 OE
- 4) N= scanner.nextInt(); ← 2 OE
- 5) for(i = 0; i < N; i++) ← 1 OE, 1 OE, 2 OE
- 6) for(j=0; j < N; j++) ← 1 OE, 1 OE, 2 OE
- 7) suma += i * j; ← 3 OE
- 8) System.out.println("El resultado es: ", suma); ← 1 OE

Análisis de Algoritmos

Estructura de Datos

35

- 1) Scanner scanner = new Scanner(System.in); ← 2 OE
- 2) int i, j, N, suma = 0; ← 1 OE
- 3) System.out.println("Ingrese el valor de N: "); ← 1 OE
- 4) N= scanner.nextInt(); ← 2 OE
- 5) for(i = 0; i < N; i++) ← 1 OE, 1 OE, 2 OE
- 6) for(j=0; j < N; j++) ← 1 OE, 1 OE, 2 OE
- 7) suma += i * j; ← 3 OE
- 8) System.out.println("El resultado es: ", suma); ← 1 OE

Corresponde al for externo

Corresponde al for interno

$$T(n) = 2 + 1 + 1 + 2 + \sum_{i=0}^{N-1} \left(1 + 1 + \sum_{j=0}^{N-1} (3 + 1 + 2) + 1 + 2 \right) + 1$$

$$T(n) = 9 + \sum_{i=0}^{N-1} \left(5 + \sum_{j=0}^{N-1} (6) \right)$$

$$T(n) = 9 + \sum_{i=0}^{N-1} (5 + (n-1-0+1) * 6)$$

$$T(n) = 9 + \sum_{i=0}^{N-1} (5 + 6n)$$

$$T(n) = 9 + (n-1-0+1) * (5 + 6n)$$

$$T(n) = 9 + 5n + 6n^2$$

$$T(n) = 6n^2 + 5n + 9$$

O(n) → Complejidad Cuadrática

Recordar que la comparación e incremento/decremento del 1er. FOR van dentro de la 1er. sumatoria

Análisis de Algoritmos

Estructura de Datos

36

Ejemplos

1- Calcular el T(N)

```

1) void imprimir() {
2)   System.out.println("1"); ← 1 OE
3)   System.out.println("2"); ← 1 OE
4)   System.out.println("3"); ← 1 OE
5)   System.out.println("4"); ← 1 OE
6)   System.out.println("5"); ← 1 OE
7)   System.out.println("6"); ← 1 OE
8)   System.out.println("7"); ← 1 OE
9)   System.out.println("8"); ← 1 OE
10) }

```

8 OE

T(n) = 8 OE
0(1) → Complejidad Constante

2- Calcular el T(N)

```

1) #include <stdio.h>
2) int main(int argc, char **argv) 3) {
3)     int i, n;
4)     scanf("%d",&n); ← 2 OE
5)     for (i = 0; i <= n; i++) { ← 1 OE 1 OE 2 OE
6)         printf("%d\n",i); ← 2 OE
7)     }
8)     return 0; ← 1 OE
9) }

```

$$T(n) = 2 + 1 + 1 + \sum_{i=0}^N (2 + 1 + 2) + 1$$

$$T(n) = 5 + \sum_{i=0}^N (2 + 1 + 2)$$

$$T(n) = 5 + \sum_{i=0}^N 5$$

$$T(n) = 5 + (n - 0 + 1) * 5$$

$$T(n) = 5 + 5n + 5$$

$$T(n) = 5n + 10$$

O(n) → Complejidad Lineal

Consultas...

¿?